

指针/结构体练习

前置任务1——特殊的数组之指针数组

学习完了内存方面的知识，你知道变量不过是储存在内存中的一段数据而已。而我们又知道数组是一段连续存放相同类型数据的内存，是不是意味着只要有类型有数据就能够组成一个数组进行存取呢？也就是说，我们可以让任何数据类型都能够组成数组，而不仅限于整数数组或字符数组（字符串）。

我们现在就要学习一个特殊的数组——指针数组，同时提供一个练习题目。

顾名思义，这是个存储指针的数组。我们以 `int*` 类型的指针为例，学习如何使用指针数组。

代码块

```
1  int a[10] = {}, b[2] = {1,2}, c[10] = {0,1,2,3,4,5,6,7,8,9};
2  int* parr[10];
```

以上代码段定义了一个指针数组，最多能够存放10个 `int*` 类型的指针。除此以外还定义了三个普通的 `int` 类型数组。利用刚刚学过的知识，我们可以这样初始化这个指针数组。

代码块

```
1  int a[10] = {}, b[2] = {1,2}, c[10] = {0,1,2,3,4,5,6,7,8,9};
2  int* parr[10];
3  parr[0] = a;
4  parr[1] = b;
5  parr[0] = c;
```

我们把三个数组的首地址存进了这个指针数组，而三个数组又有着各自不一样的地方，这就是指针数组的妙用。通过指针数组，我们能够将指向不同内存段的指针放到一个更好获取的位置，只通过 `parr` 就能够访问这三个数组的元素。例如：

代码块

```
1  int a[10] = {}, b[2] = {1,2}, c[10] = {0,1,2,3,4,5,6,7,8,9};
2  int* parr[10];
3  parr[0] = a;
4  parr[1] = b;
5  parr[2] = c;
6
```

```
7 printf("parr[2][9] = %d\n", parr[2][9]);
```

你会发现这居然还和二维数组有点相似之处！不过二维数组要求每行的元素数量必须相等，而且内存实际是连续分布的。

用指针数组实现特殊的二维数组

模仿上面的代码，用指针数组实现二维数组。要求如下：

1. 二维数组共有三行：第一行可容纳2个元素，第二行可容纳3个元素，第三行可容纳4个元素
2. 为该二维数组赋值如下值
 - a. 1, 2
 - b. 3, 4, 5
 - c. 6, 7, 8, 9
3. 最后按上面的格式进行输出，注意逗号（半角符号逗号）后有空格

不限其他完成要求，代码尽量简洁易读即可。

完成后将你的源代码重命名为 `2d_array` 并放入 `姓名-专业班级-Great-3` 文件夹，等待作业提交。

前置任务2——特殊的数组之字符串

大家应该已经学习过了字符数组，至少已经知道了一部分内容，但这些其实都是属于数组的内容。

像这样声明的数组我们称为 **字符数组**：

代码块

```
1 char arr[10];
```

字符数组可以进行如下的初始化予以初始值，其中 `'x'` 表示字符x：

字符数组

```
1 char arr[10] = {'H', 'a', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd'};  
2 char ch = '?'; // 与char对比
```

什么是本质？

与本质相对的名词是现象，本质与现象分别揭示了事物的内在联系和外在表现。

本质是事物的根本性质，是事物之间存在的内在联系，具有一般的、普遍的特性。

和本质不同，现象是个别的、具体的。

这两对相反的性质说明我们可以利用相似的方法来解决本质相同的问题，也可以用特殊的办法去解决现象上不同的问题。

理解本质的概念有助于我们继续深入学习字符串的知识

字符串是字符数组的一种**特殊**情况。具体来说，字符串相比字符数组增加了结束的标志，也就是 `'\0'`，其值为0，我们称之为终止符。既然字符串被包含于字符数组，那么**字符串的本质就依然是数组**。

printf是如何输出字符串的

我们前面学过，`printf` 可以利用 `%s` 占位符来输出一个字符串。但为什么不能是一个普通的字符数组呢？

答案就在 `'\0'` 中。我们既然已经学过指针，那么就用指针的办法来理解。以下是示例代码：

代码块

```
1 char arr[10] = {'H', 'a', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'd', '.'};
2 printf("%s", arr);
```

此处第二行的 `arr` 会被编译器解释成指向 `arr` 数组首元素的指针。

当 `printf` 检查到 `%s` 占位符后，我们传入的参数又是指向一个字符元素的指针，那么就会从该指针指向的字符开始向后逐个输出字符。此时的现象就是输出 `Halo,word.`。然而我们提供的代码似乎有些问题，它的输出可能并不符合我们的预期，这是为什么？

答案就是 `arr` 并不是一个字符串，而是一个普通的字符数组。字符串必须有一个 `'\0'` 作为终止，`printf` 就是通过判断终止符号来决定是否停止输出。若没有检查到终止符，`printf` 就会不断输出字符直到找到一个可能的终止符。

更深入更苦难的知识可以从此了解：[printf如何输出（知识扩展）](#)

字符串

在清楚地了解了字符串和字符数组的区别与联系后，我们终于能够正式开始对字符串的学习。

C语言中，我们可以用字符数组来储存字符串，就像如下代码中的`arr`数组便储存了一个 `Halo, World!` 字符串：

代码块

```
1 char arr[20] = {'H', 'a', 'l', 'o', ',', ' ', 'W', 'o', 'r', 'l', 'd', '!', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0'};
```

```
'\0', };
```

请注意不要丢失了终止符——尽管在该数组进行这种初始化后，那些未被你主动初始化的元素会被默认赋值为0。

同时还有另一种初始化方法，语法如下：

代码块

```
1 char arr[20] = "Halo, word!";
```

[其原理可以点击本链接进行了解。](#)

还有两个概念需要我们了解：字符串的长度和大小。

对于一个字符数组来说，讨论长度和大小并没有意义，因为数组的长度和大小在定义时便已经确定且相等，但对于其中的字符串则不同。

我们设字符串的大小为 `N`，其中包括了字符串的结束——`'\0'`。而字符串的长度就是 `N-1`，并不包含 `'\0'`。例如 `"Halo"` 中包含了4个字符和一个 `'\0'`，那么这里的 `N` 就等于5。[稍微详细的解释也可以点击本链接了解。](#)

同时还有许多和字符串/字符数组的处理有关的C语言库函数，像是`strcpy()`，`strcmp()`，`strcat()`等，需要你们自主STFW进行学习。

我们也完全可以利用数组之间的相同本质来解决问题，仅仅需要处理的是 `char` 类型和其他类型所占字节大小的不同。这样来看，这些库函数也不过是实现得好的方法，如果遇到了并不是特别适合使用它们的情况，自己去实现或许要比强行使用它们更好。

练习

自制库函数

C语言对于字符数组有许多相关的函数，并且由于字符类型的大小恰好为1，字符数组还经常用来存储一些二进制数据。

你需要实现一个字符串复制函数，要求如下：

1. 函数名为 `my_strcpy`，返回值为 `int` 类型
2. 能接收两个参数，均为 `char*` 的指针类型
 - a. 第一个参数为指向复制源的指针
 - b. 第二个参数为指向复制目标的指针
3. 将复制源指向的元素作为起始，将数据覆盖复制到复制目标处，直到复制到目标处的字符为 `'\0'`

4. 最后返回复制的字符数（字节数），包含最后的 `'\0'`

这样我们就实现了一个类似 `strcpy` 的函数，只是返回值不同。如果你想实现其他类似的函数，也可以自己动手实现。

最后将含有你的自制函数的文件重命名为 `string`，放入 `姓名-专业班级-Great-3` 文件夹，等待作业提交。

字符串字面量（知识扩展）

[字符不同于数字](#)，同时为了方便地初始化（initialization），C语言提供了 **字符串字面量**（String literals）来方便我们进行初始化。字符串我们都知道是什么，而字面量就是指用我们可以直接在代码里写出来（字面意义上的）并用来表示某种东西的值，字符串字面量就是说我们可以用它像表示普通变量一样对字符串进行表示。

语法如下：

代码块

```
1 char arr[10] = "Halo.";
```

其中 `=` 后面用双引号包裹起来的被我们就称之为 **字符串字面量**，[它是一段以 `'\0'` 结尾的字符序列](#)。

我们也可以将它看做是一个有 `6` 个元素的字符数组，里面有一个字符串，它的成员如下：

代码块

```
1 'H', 'a', 'l', 'o', '.', '\0'
```

由于 `'\0'` 在字符串中并不表示一个有效的信息，仅仅表示其结束位置，但又必须占据一个元素的空间，所以这个 `6` 被称为**该字符串**的大小，而 `6 - 1` 被称为**该字符串**的长度。

因此，在 `[]` 中的数字必须要足够得大，以存下这里面的元素。在C语言中，这个数字不能小于 `=` 后方字符串的长度，否则将会编译失败。

如果只想让字符数组恰好能够存储这一个字符串，那么 `[]` 内的数字可以省略。

注释：

1. 字符不同于数字：高效地用二进制存储信息

- a. 对于数字来说，数字12就是会被存储为12，而不会被存储为1, 2，这是因为12相比1, 2更加节省存储空间（一个 `int` 变量和两个 `int` 变量的区别），同时也更加利于计算。

b. 但字符不同，计算机想要储存字符就要依赖编码（以下仅以ASCII码为例），储存abc时只能存储a, b, c，而不能直接存储abc，这是因为abc不能和a用一样的类型（`char`）进行表示。

i. 字符数组的类型不是 `char` 类型。

2. 以 `'\0'` 结尾的字符序列

a. 对于任何 `字符串字面量` 表示，我们都不需要手动添加 `'\0'` 来结尾。在编译的某个阶段，编译器会为它自动加上 `'\0'`。

b. 同时其定义只要求结尾是 `'\0'`，也就是说无论我们在中间有多少 `'\0'` 都不会影响到整体的字符数组初始化。

c. 同样地，结合字符串的定义，这也说明一个 `字符串字面量` 可以包含多个字符串，这样我们就能将它与字符串进行区分。

printf如何输出（知识扩展）

让我们从头开始吧。

我们前面学过，`printf` 可以利用 `%s` 占位符来输出一个字符串。但为什么不能是一个普通的字符数组呢？

答案就在 `'\0'` 中。我们既然已经学过指针，那么就用指针的办法来理解。以下是示例代码：

代码块

```
1 char arr[10] = {'H', 'a', 'l', 'o', ',', 'w', 'o', 'r', 'd', '.'};
2 printf("%s", arr);
```

这里第二行的 `arr` 将会被编译器解释为指向 `arr` 数组首个元素的指针，类型为 `char *`。

当 `printf` 检查到 `%s` 占位符后，我们传入的参数又是指向字符类型元素的指针（`char *`），那么就会从该指针指向的字符开始向后逐个输出字符。其现象就是输出 `Halo,word.`。然而我们提供的代码似乎有些问题，但它的输出却仍然可能符合我们的预期，为什么？这其实还涉及到了更深入的内存知识，所以暂时不对这一点进行探讨。但我们却可以通过后面的示例代码来向你讲解其输出的原理。

我们给出如下代码供你编译运行，在这之后你需要更改代码来让程序的输出符合预期。

示例1：

代码块

```
1 char arr[10] = {'H', 'a', 'l', 'o', ',', 'w', 'o', 'r', 'd', '.'}; // 虽然没有终止符，但后面一定会有值为0的一段内存
2 char ch = '?';
3 printf("%s\n", &ch); // 这里我们想通过 %s 来输出 ch 字符
```

```
4  /* 这里我们想要输出的是如下结果
5  ?
6  */
```

示例2:

代码块

```
1  char arr[10] = {'H', 'a', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'd', '.'};
2  char arr2[10] = {'\n', 'H', 'i', '\0'};    // 这里有终止符
3  printf("%s\n", arr2);
4  printf("%s\n", arr);
5  /* 这里我们想要输出的是如下结果
6
7  Hi
8  Halo,word.
9  */
```

造成错误的原因就是我们给出的 `arr` 里面储存的并不是一个字符串，它只是一个普通的字符数组。字符串必须有一个 `'\0'` 作为终止，`printf` 就是通过判断终止符号来决定是否停止输出。若没有检查到终止符，`printf` 就会不断输出字符直到找到一个可能的终止符。

出现这个的原因如下：

例1中，`arr` 字符数组在内存中位于 `ch` 变量后，且中间没有任何中间地址（在内存中他们是连续分布的）。那么 `printf` 输出 `ch` 后就会直接读取并输出 `arr`。

例2中，`arr2` 字符数组在内存中的位于 `arr` 字符数组后，且中间也没有任何中间地址。那么 `printf` 输出 `arr` 后就会继续读取并输出 `arr2`。

而为什么输出结果会是这样？为什么在第一个示例中 `ch` 后面跟着 `arr`？为什么第二个示例中 `arr` 后面又是 `arr2`？在未来更深入地学习内存对齐知识时将会学习到，我们暂且不进行讨论。

修改代码

你需要修改以上两个实例代码来使得 `printf` 输出其本应该输出的东西。